Data Structures (ETCCDS202)

Unit–4 Assignment: Non-Linear & Advanced Data Structures

Name of the School: School of Engineering & Technology

Program : B.Tech (CSE / AIML)

Section : A

Student Name : Disha Saini

Roll No. : 2501730318

Submitted To : Ms. Neetu

1. Introduction

In this assignment, a Data Management Mini Toolkit (DMMT) was developed using Python to demonstrate the working of important non-linear and advanced data structures. These data structures are widely used in real-world systems such as databases, search engines, and networking applications.

The toolkit includes three main components:

- Binary Search Tree (BST)
- Graph Traversal (BFS and DFS)
- Hash Table with collision handling

The objective is to understand how data is stored, searched, and managed efficiently.

2. Binary Search Tree (BST)

2.1 BST

A Binary Search Tree is a hierarchical data structure in which each node contains a key and has at most two children. The defining property of a BST is that all values in the left subtree are smaller than the root, and all values in the right subtree are greater. This property allows efficient searching and ordered storage of data.

In this assignment, the BST was implemented with operations such as insertion, searching, deletion, and inorder traversal.

2.2 Operations

Operations Implemented

- Insertion
- Searching
- Deletion (all 3 cases)
- Inorder Traversal

During insertion, each new value is placed in its correct position based on comparisons, ensuring that the BST property is maintained. The search operation follows a similar logic, moving left or right depending on the value being searched.

Deletion in a BST is slightly more complex because it involves three different cases. If the node to be deleted is a leaf node, it can simply be removed. If the node has only one child, it is replaced by its child. In the case where the node has two children, it is replaced with its inorder successor, which is the smallest value in its right subtree. This ensures that the tree remains valid after deletion.

Inorder traversal is an important operation in BSTs because it visits nodes in the sequence left, root, and right. Due to the structure of the BST, this traversal always produces values in sorted order.

Inorder traversal follows:
Left → Root → Right

2.3 Time Complexity

In terms of performance, BST operations such as insertion, search, and deletion have an average time complexity of O(log n). However, in the worst case, when the tree becomes skewed, the complexity can degrade to O(n).

2.4 Applications

BSTs are commonly used in applications such as database indexing, searching systems, and maintaining sorted data efficiently.

3. Graph

3.1 Introduction

A graph is a data structure used to represent relationships between different entities. In this assignment, a directed and weighted graph was implemented using an adjacency list. In this representation, each node maintains a list of its neighboring nodes along with the weights of the connecting edges, making it space-efficient and suitable for sparse graphs.

Two important graph traversal algorithms were implemented: Breadth First Search (BFS) and Depth First Search (DFS). These algorithms are used to explore all nodes in a graph systematically.

3.2 Breadth First Search (BFS)

Breadth First Search (BFS) works by visiting nodes level by level, starting from a given source node. It uses a queue data structure to keep track of nodes that need to be visited next. BFS ensures that all nodes at the current level are explored before moving to the next level. This property makes BFS particularly useful for finding the shortest path in unweighted graphs and for applications like network broadcasting.

3.3 Depth First Search (DFS)

Depth First Search (DFS), on the other hand, explores a graph by going as deep as possible along one path before backtracking. It is typically implemented using recursion or a stack. DFS is useful in problems such as cycle detection, path finding, and solving puzzles like mazes.

3.4 Difference between BFS and DFS

The main difference between BFS and DFS lies in their traversal strategy. BFS explores nodes in a level-wise manner, while DFS explores nodes in a depth-wise manner. Both have their own advantages depending on the problem being solved.

4. Hash Table with Separate Chaining

4.1 Introduction

A hash table is a data structure that provides fast access to data using a hash function. The hash function maps a key to a specific index in an array, allowing quick insertion, retrieval, and deletion of values.

4.2 Hash Function

In this assignment, a simple hash function was used, where the index is calculated using the modulo operation. However, multiple keys can sometimes map to the same index, which leads to a situation known as a collision.

Hash Function Used -

```
index = key % table_size
```

4.3 Collisions

A collision occurs when multiple keys map to the same index.
Example:

- 10 % 5 = 0
- 15 % 5 = 0
- 20 % 5 = 0

To handle collisions, the technique of separate chaining was used. In this method, each index of the hash table stores a list (or chain) of key-value pairs. When a collision occurs, the new element is simply added to the list at that index instead of overwriting existing data. This ensures that all values are preserved and can still be accessed correctly.

4.4 Operations Implemented

The hash table implemented in this assignment supports insertion, retrieval, and deletion operations.

4.5 Time Complexity

On average, these operations take constant time, O(1), making hash tables very efficient. However, in the worst case, when many collisions occur, the time complexity can degrade to O(n).

- Average: O(1)

- Worst Case: O(n) (many collisions)

4.6 Applications

Hash tables are widely used in applications such as caching systems, dictionaries, database indexing, and fast lookup operations.

5. Results and Output

The implemented Data Management Mini Toolkit was executed successfully, and the outputs obtained confirm the correct working of all modules.

A sample of the program output is shown below:

```
=========== BST TEST ===========
Initial Inorder:
20 30 40 50 60 70 80
Search 20: True
Search 90: False

Delete leaf node (20):
30 40 50 60 70 80

Insert 65 and delete 60 (one child case):
30 40 50 65 70 80

Delete node with 2 children (30):
40 50 65 70 80
```

In the Binary Search Tree module, the inorder traversal initially produced a sorted sequence of elements, which verifies the correctness of insertion. After performing deletion operations for all three cases (leaf node, one child, and two children), the inorder traversal continued to produce a correctly sorted sequence, confirming that the BST properties were maintained.

In the graph module, the adjacency list representation was printed correctly, showing all nodes along with their connected edges and weights. The BFS traversal from node A visited nodes level by level, while the DFS traversal explored nodes in a depth-first manner. The traversal outputs were consistent with the expected behavior of BFS and DFS algorithms.

In the hash table module, collisions were successfully demonstrated using keys that mapped to the same index. The use of separate chaining allowed multiple key-value pairs to be stored in the same bucket. Retrieval operations returned correct values, and deletion from a collided bucket updated the structure accurately.

Overall, the outputs validate that all components of the toolkit are functioning as expected.

6. Observations

During the implementation of this assignment, it was observed that different data structures are suitable for different types of problems. The Binary Search Tree is efficient for maintaining sorted data and performing search operations, but its performance can degrade if the tree becomes unbalanced.

Graph traversal algorithms such as BFS and DFS provide different ways of exploring data. BFS is more suitable for shortest path problems in unweighted graphs, whereas DFS is useful for exploring deeper relationships and detecting cycles.

The hash table demonstrated very fast data access under normal conditions. However, it was also observed that collisions can affect performance, which highlights the importance of a good hash function and proper collision handling techniques.

7. Conclusion

This assignment provided practical experience in implementing and understanding important data structures used in computer science. The Binary Search Tree demonstrated efficient ordered storage and searching, the graph algorithms illustrated how relationships and networks can be explored, and the hash table showed how fast data access can be achieved even in the presence of collisions. Overall, these data structures form the foundation of many real-world systems, and understanding them is essential for building efficient and scalable applications.

8. References

- Class notes from Data Structures
- Lecture PPTs
- GeeksforGeeks